

University of Cape Town
Department of Electrical Engineering

EEE3096S Practical 1B

Execution Timing and Cycle-Accurate Assembly on the UCT Dev Board

Student 1: Thapelo Mocheko (MCHTHA046)
Student 2: Velan Pillay (PLLVEL003)
Year: 2026

Report filename required by the practical:
prac1B_MCHTHA046_PLLVEL003.pdf

Contents

1	Golden Measure and PC Timing – Task 1	3
1.1	Method and predicted execution time	3
1.2	Timing runs	3
1.3	Golden outputs	3
1.4	Hand check	3
1.5	Task 1 question	4
2	Board Implementation and Timer Measurement – Task 2	4
2.1	Clock path and timer configuration	4
2.2	Predicted timing	4
2.3	Measured single-call timing	4
2.4	Counter wrap-around	5
2.5	Golden-value verification	5
2.6	Task 2 question	5
3	Optimisation Flags – Task 3	5
3.1	Prediction	5
3.2	Optimisation results	6
3.3	Disassembly evidence	7
3.4	Execution-time/code-size trade-off	7
3.5	Task 3 question	8
4	Cycle-Counted Phase Delay – Task 4	8
4.1	Required phase delay	8
4.2	Assembly implementation	8
4.3	Cycle budget	9
4.4	Measured error and discussion	9
5	Analog Debugging of the LCD Enable Line – Task 5	10
5.1	4-bit LCD interface	10
5.2	Uncorrected Enable timing	10
5.3	Measured rise time and parasitic capacitance	11
5.4	NOP padding calculation	12
5.5	Fixed timing evidence	12
5.6	LCD result	13
	References	14

A	Code Appendix	15
A.1	Task 1 PC golden implementation	15
A.2	Tasks 2–3 board C implementation	15
A.3	Task 4 assembly	15
A.4	Task 5 assembly	15
B	Submission Evidence Checklist	16

1 Golden Measure and PC Timing – Task 1

1.1 Method and predicted execution time

Describe the PC implementation of the golden integer square-root function using double-precision arithmetic and the standard-library square-root function. State the instruction-count estimate and the PC CPU clock used before running the timing experiment.

TO FILL: CPU model and clock frequency; estimated instruction count; predicted time per call in ns; brief reasoning.

1.2 Timing runs

The practical requires two timing runs with different repetition counts and a comparison of the resulting time per call.

Table 1: Task 1 timing runs.

Run	Repetitions	Total measured time	Time/call (ns)	Notes
1				
2				

TO FILL: Mean time per call and spread across the two runs. Define clearly how “spread” was calculated.

1.3 Golden outputs

Complete the full table for the ten required inputs.

Table 2: Golden integer square-root outputs.

Input x	Golden output $\lfloor \sqrt{x} \rfloor$	Verification/status
0		
1		
15		
16		
4095		
65535		
123456789		
987654321		
4294836225		
4294967295		

1.4 Hand check

Write one check in full by proving both inequalities

$$y^2 \leq x \quad \text{and} \quad (y+1)^2 > x.$$

TO FILL: Choose one required input and show the complete arithmetic.

1.5 Task 1 question

TO FILL: State the golden output for 987654321, the measured time per call in ns, the repetition count used, and explain why timing one isolated call is unreliable.

2 Board Implementation and Timer Measurement – Task 2

2.1 Clock path and timer configuration

Name every clock-tree stage from HSI to the selected 16-bit timer, including divider values and the timer prescaler.

TO FILL: Timer used; HSI frequency; SYSCLK/AHB/APB path; APB timer-clock rule if applicable; prescaler value; resulting counter frequency and counter tick period. Cite the exact RM0091 sections used.

2.2 Predicted timing

TO FILL: Predicted instruction count, predicted counter span, and predicted execution time for one integer square-root call. Show reasoning.

2.3 Measured single-call timing

$$\Delta N = \text{counter span} = \boxed{}, \quad (1)$$

$$f_{\text{counter}} = \boxed{} \text{ Hz}, \quad (2)$$

$$\Delta t = \frac{\Delta N}{f_{\text{counter}}} = \boxed{} \mu\text{s}. \quad (3)$$

TO FILL: Insert the actual counter values before/after the call and show the complete divider-to-microseconds calculation.

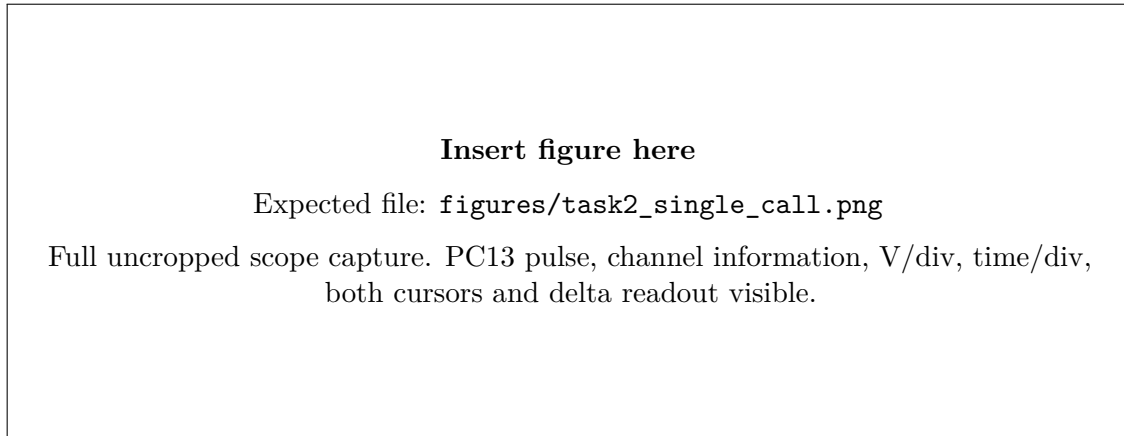


Figure 1: Task 2 single-call execution pulse on PC13. The body text must state the same measured pulse width as the cursor readout.

2.4 Counter wrap-around

For a 16-bit free-running counter, one wrap occurs after $2^{16} = 65536$ counts. Show the long-run arithmetic explicitly rather than assuming unsigned subtraction without explanation.

TO FILL: Long-run repetition count; start and end counter values; number of wraps; total count span; total time; mean time per call.

2.5 Golden-value verification

TO FILL: State pass/fail against all ten Task 1 golden values and include the evidence produced by the firmware.

2.6 Task 2 question

TO FILL: State the scope pulse width in us and the recorded counter span. Show the full calculation from divider values to pulse width. State whether the system clock matches the 8 MHz HSI and explain how this was confirmed.

3 Optimisation Flags – Task 3

3.1 Prediction

TO FILL: Predicted ratio between the -O0 and -O2 execution times, written down before building -O2, with brief reasoning.

3.2 Optimisation results

Table 3: Effect of compiler optimisation on execution time and text size.

Flag	Text size (bytes)	Counter time/call (μ s)	Scope width (μ s)	Ratio to -O0
-O0				1.000
-O1				
-O2				
-Os				

Table 4: Predicted and measured optimisation ratios.

Comparison	Predicted ratio	Measured ratio
-O0 / -O2		

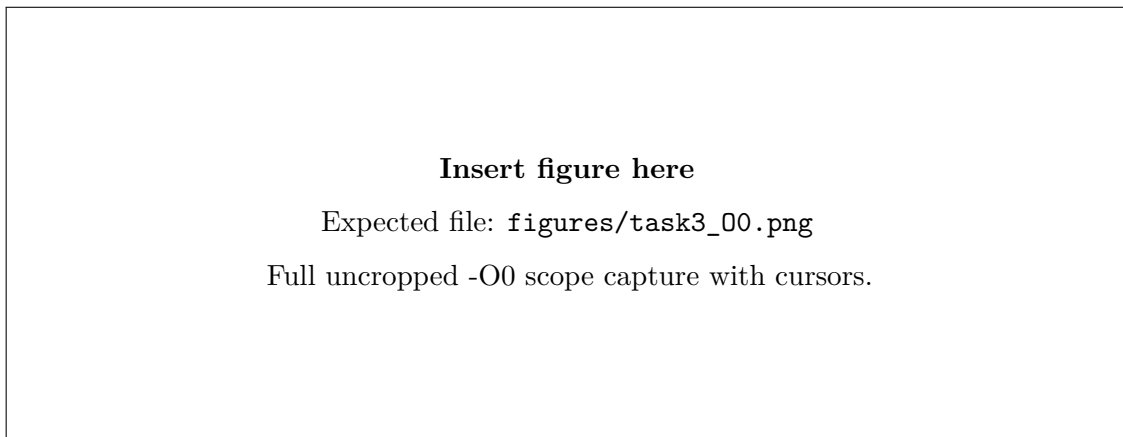


Figure 2: Task 3 execution pulse at -O0.

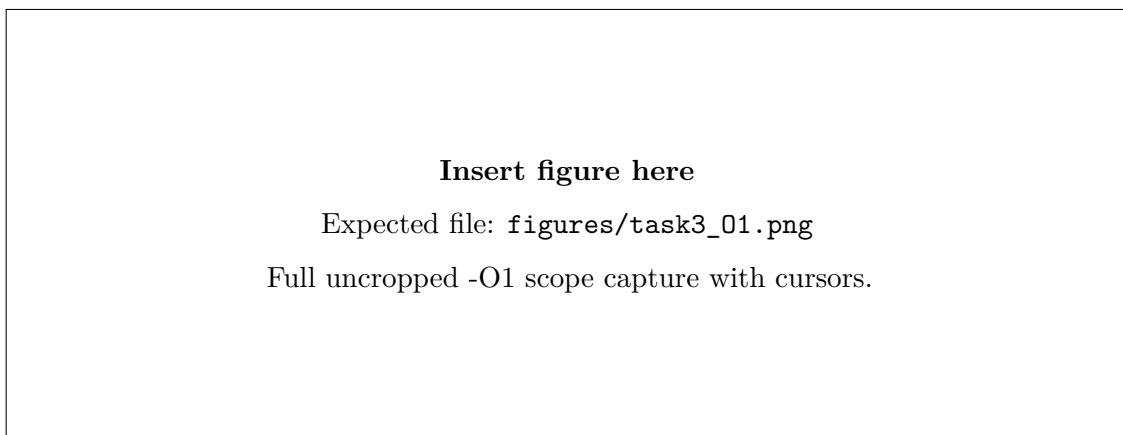


Figure 3: Task 3 execution pulse at -O1.

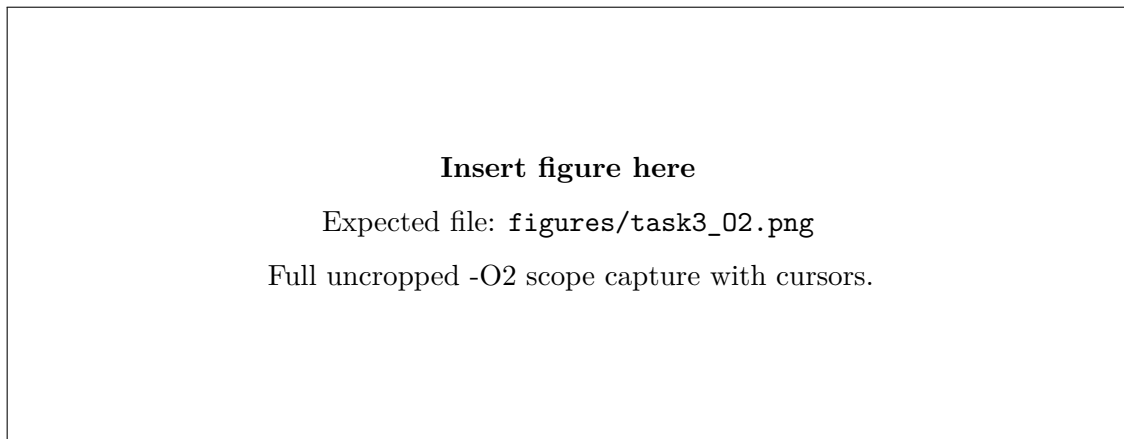


Figure 4: Task 3 execution pulse at -O2.

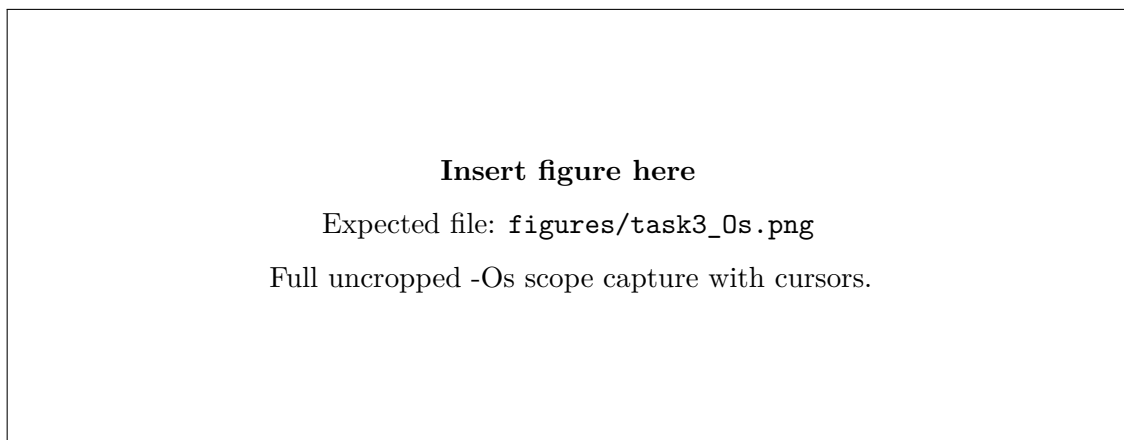


Figure 5: Task 3 execution pulse at -Os.

3.3 Disassembly evidence

Name one optimisation transformation visible in the -O2 disassembly and identify the source lines on which it acts.

TO FILL: Name of transformation, source-line numbers, and a short explanation of what changed. Paste a concise disassembly excerpt below.

Listing 1: Task 3 disassembly excerpt at -O2.

1 % Paste only the relevant disassembly excerpt here.

3.4 Execution-time/code-size trade-off

TO FILL: Explain the measured trade-off between execution time and text size across -O0, -O1, -O2 and -Os.

3.5 Task 3 question

TO FILL: State measured -O0 and -O2 times in us; measured ratio; predicted ratio; optimisation transformation; and the source lines affected.

4 Cycle-Counted Phase Delay – Task 4

4.1 Required phase delay

The input frequency is 1 kHz, so the period is

$$T = \frac{1}{f} = \frac{1}{1000} = 1 \text{ ms.} \quad (4)$$

A 45° phase shift is one eighth of a cycle, therefore

$$\Delta t = \frac{45^\circ}{360^\circ} T = \frac{1}{8} (1 \text{ ms}) = \boxed{125 \text{ } \mu\text{s}}. \quad (5)$$

With the system clock at 8 MHz,

$$T_{\text{cycle}} = \frac{1}{8 \text{ MHz}} = \boxed{125 \text{ ns}}. \quad (6)$$

Hence the target loop period is

$$N_{\text{cycles}} = \frac{125 \text{ } \mu\text{s}}{125 \text{ ns}} = \boxed{1000 \text{ cycles}}. \quad (7)$$

4.2 Assembly implementation

The core polling loop used for the cycle-counted design is shown below. The ADC sample is read from the ADC data register and written directly to DAC1 before the software delay.

Listing 2: Core Task 4 ADC-to-DAC loop.

```

1 DSP_Loop:
2     LDR R0, =ADC_DR
3     LDR R1, =DAC_DHR12R1
4
5 loop:
6     LDR R2, [R0] @ predicted 2 cycles
7     STR R2, [R1] @ predicted 2 cycles
8
9     MOVS R3, #248 @ 1 cycle
10    NOP @ 1 cycle
11    NOP @ 1 cycle
12
13 delay_loop:
14    SUBS R3, R3, #1 @ 1 cycle x 248
15    BNE delay_loop @ 247 taken + 1 not taken
16
17    B loop @ predicted 3 cycles

```

4.3 Cycle budget

Table 5: Task 4 cycle budget for one ADC-to-DAC update.

Instruction/event	Cycles/instruction	Iterations	Subtotal	Running total
LDR R2, [R0]	2	1	2	2
STR R2, [R1]	2	1	2	4
MOVS R3, #248	1	1	1	5
NOP	1	2	2	7
SUBS	1	248	248	255
BNE taken	3	247	741	996
BNE not taken	1	1	1	997
B loop	3	1	3	1000

Thus the predicted update period is

$$1000(125 \text{ ns}) = \boxed{125 \mu\text{s}}, \quad (8)$$

which corresponds to the required 45° phase displacement at 1 kHz.

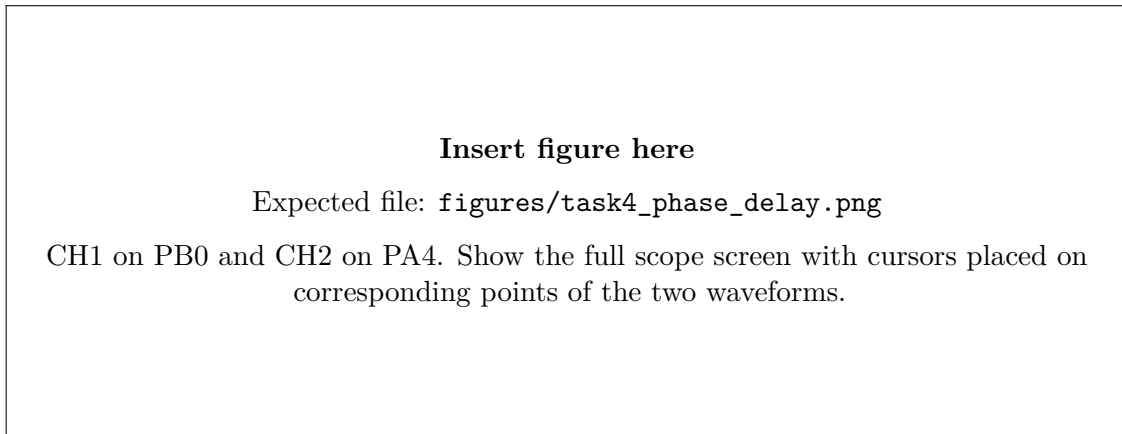


Figure 6: Task 4 input on PB0 and DAC output on PA4. Cursor separation $\Delta t = \underline{\hspace{2cm}} \mu\text{s}$, corresponding to the measured phase delay.

4.4 Measured error and discussion

Use the actual oscilloscope reading in the following calculation:

$$\Delta t_{\text{meas}} = \boxed{\hspace{2cm}} \mu\text{s}, \quad (9)$$

$$\varepsilon_t = \Delta t_{\text{meas}} - 125 \mu\text{s}, \quad (10)$$

$$\% \text{ error} = \frac{|\varepsilon_t|}{125 \mu\text{s}} \times 100 = \boxed{\hspace{2cm}} \%. \quad (11)$$

Any non-zero error should be discussed using the measured result. Plausible contributors include HSI oscillator tolerance, scope cursor placement, ADC/DAC peripheral timing and the discrete sample-and-hold nature of the DAC waveform. Do not claim a contributor that is inconsistent with the observed data.

5 Analog Debugging of the LCD Enable Line – Task 5

5.1 4-bit LCD interface

The LCD was driven in 4-bit mode using PC15 as Enable, PC14 as RS, and PA15/PA12/PB9/PB8 as D7/D6/D5/D4. The program waits for the LCD power rail, executes the 4-bit initialisation sequence, and writes ASCII 0x41 (“A”).

Listing 3: Task 5 entry point and character write.

```

1 LCD_Run:
2     PUSH {LR}
3
4     BL LCD_DelayLong
5     BL LCD_DelayLong
6     BL LCD_DelayLong
7     BL LCD_DelayLong
8
9     BL LCD_Init
10
11    MOVS R0, #0x41
12    BL LCD_WriteData
13
14 hang:
15    B hang

```

The 4-bit power-up sequence used the standard pattern of three 0x3 nibbles followed by 0x2, after which full commands such as 0x28, 0x08, 0x01, 0x06, 0x0C and 0x80 were sent through the 4-bit write routine.

5.2 Uncorrected Enable timing

Before adding NOP padding, the PC15 HIGH and LOW masks were preloaded so that no unintended literal-load instruction lengthened the HIGH pulse:

Listing 4: Uncorrected Enable pulse used for the failing-case measurement.

```

1 LCD_Pulse:
2     PUSH {R0, R1, R2, LR}
3     LDR R0, =GPIOC_BSRR
4     LDR R1, =0x00008000 @ PC15 HIGH mask
5     LDR R2, =0x80000000 @ PC15 LOW mask
6
7     STR R1, [R0] @ PC15 HIGH
8     @ no NOP padding here
9     STR R2, [R0] @ PC15 LOW
10
11    BL LCD_DelayShort
12    POP {R0, R1, R2, PC}

```

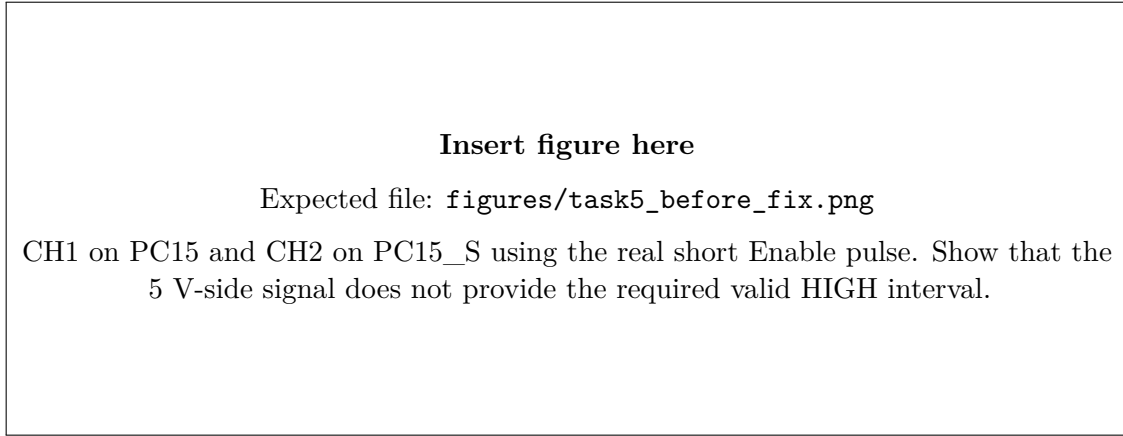


Figure 7: Task 5 before-fix Enable waveform: PC15 (CH1) and PC15_S (CH2).

5.3 Measured rise time and parasitic capacitance

A long diagnostic square wave was used only to make the RC rising edge easy to capture. The measured time from approximately 0 V to the 3.5 V threshold on PC15_S was approximately

$$t_r \approx 27 \text{ ns}. \quad (12)$$

Important: replace 27 ns everywhere in this report if the final cursor placement at exactly 3.5 V gives a different value.

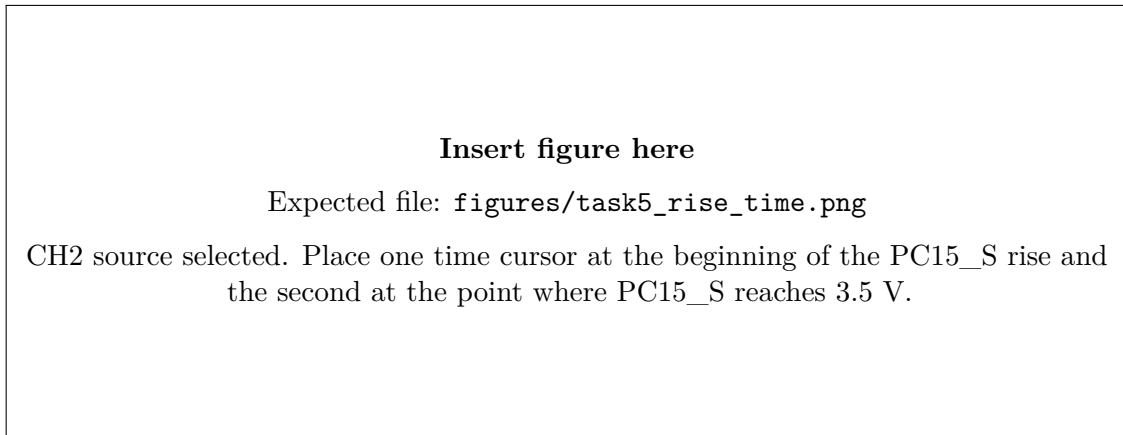


Figure 8: Measured PC15_S rise time from 0 V to 3.5 V: $t_r = \underline{\hspace{2cm}}$ ns.

The practical gives the capacitor-charging relation

$$V_C(t) = V_{CC} \left(1 - e^{-t/(RC)} \right), \quad (13)$$

which gives

$$C = \frac{-t_r}{R \ln \left(1 - \frac{V_C(t)}{V_{CC}} \right)}. \quad (14)$$

For this board, substitute

$$t_r = 27 \text{ ns}, \quad V_C(t) = 3.5 \text{ V}, \quad R = \underline{\hspace{1cm}} \Omega, \quad V_{CC} = \underline{\hspace{1cm}} \text{ V}.$$

Therefore,

$$C = \underline{\hspace{1cm}} \text{ pF}. \quad (15)$$

The resistor value and supply voltage must be taken from the actual board/schematic and bench measurement; they are intentionally not invented here.

5.4 NOP padding calculation

At 8 MHz, each CPU cycle is 125 ns. Using the measured rise time of approximately 27 ns and the practical requirement that PC15_S remain above 3.5 V for at least 450 ns, a conservative required MCU-side HIGH time is

$$t_{\text{required}} = t_r + 450 \text{ ns} \approx 27 \text{ ns} + 450 \text{ ns} = 477 \text{ ns}. \quad (16)$$

Hence

$$N_{\text{NOP}} = \left\lceil \frac{477 \text{ ns}}{125 \text{ ns}} \right\rceil = \lceil 3.816 \rceil = \boxed{4 \text{ NOPs}}. \quad (17)$$

The four NOPs provide a nominal padding of

$$4(125 \text{ ns}) = \boxed{500 \text{ ns}}. \quad (18)$$

Listing 5: Corrected Enable pulse with four NOPs.

```

1  LDR R1, =0x00008000
2  LDR R2, =0x80000000
3  STR R1, [R0] @ PC15 HIGH
4
5  NOP
6  NOP
7  NOP
8  NOP
9
10 STR R2, [R0] @ PC15 LOW

```

5.5 Fixed timing evidence

The final oscilloscope measurement must be made using the corrected *real* LCD Enable pulse, not the long diagnostic square wave. Place the cursors on the rising and falling 3.5 V crossings of PC15_S.

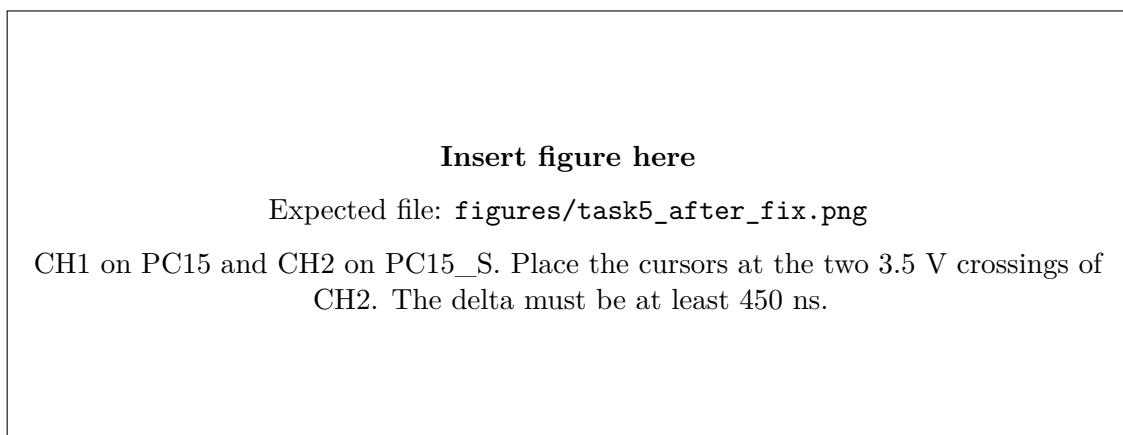


Figure 9: Task 5 fixed Enable waveform. PC15_S remains above 3.5 V for $\Delta t = \underline{\hspace{2cm}}$ ns, satisfying the 450 ns requirement.

Record the final result explicitly:

$$t_{\text{PC15_S} > 3.5\text{V}} = \underline{\hspace{2cm}} \text{ ns} \geq 450 \text{ ns}. \quad (19)$$

5.6 LCD result

The final code writes ASCII 0x41; using the working LCD module, the character “A” was displayed clearly.

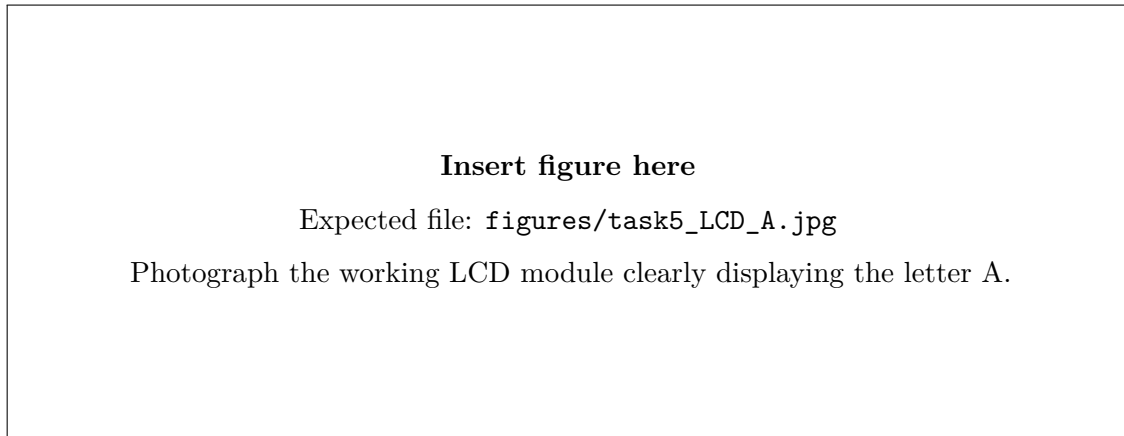


Figure 10: Final LCD output displaying the character “A”.

References

1. EEE3096S Practical 1B, *Execution Timing and Cycle-Accurate Assembly on the UCT Dev Board*, UCT, 2026.
2. STMicroelectronics, *RM0091 STM32F0x1/STM32F0x2/STM32F0x8 Reference Manual*. **TO FILL:** exact clock-tree and timer sections used in Tasks 2–3.
3. STMicroelectronics, *STM32F051x4/x6/x8 Datasheet*. **TO FILL:** exact tables/sections used.
4. Hitachi, *HD44780U LCD Controller/Driver Datasheet*. Relevant items include the 4-bit initialisation flow and bus timing characteristics. **TO FILL:** exact figure/table/page references used in the final report.
5. UCT Dev Board schematic and pinout reference. **TO FILL:** schematic sheet/reference used for the level-shifter resistor value and signal names.
6. **TO FILL:** PC CPU documentation/source used for the Task 1 CPU clock, if cited.

A Code Appendix

The practical requires the report to be followed by a code appendix. Put the final source files in a local `code/` directory beside this $\text{\LaTeX}$ file before compiling.

A.1 Task 1 PC golden implementation

Copy the final source file to `code/task1_golden.c` before compiling the submission.
Complete Task 1 golden-measure source.

A.2 Tasks 2–3 board C implementation

Copy the final source file to `code/task2_3_main.c` before compiling the submission.
Complete board implementation and timing code used for Tasks 2 and 3.

A.3 Task 4 assembly

Copy the final source file to `code/dsp.s` before compiling the submission.
Final Task 4 `dsp.s`.

A.4 Task 5 assembly

Copy the final source file to `code/lcd.s` before compiling the submission.
Final Task 5 `lcd.s`.

B Submission Evidence Checklist

Before generating the final PDF, confirm all of the following:

- The report has five numbered sections in the required order, followed by references and the code appendix.
- Task 2 has one full-screen scope capture of the PC13 single-call pulse.
- Task 3 has four full-screen captures: -O0, -O1, -O2 and -Os.
- Task 4 has one full-screen capture with CH1=PB0, CH2=PA4 and cursors on the measured phase delay.
- Task 5 includes before-fix and after-fix Enable captures; keep a separate rise-time capture as well if it is used for a separate numerical claim.
- Every scope image is uncropped and shows channel numbers, V/div, time/div, both cursors and the delta readout.
- Every timing value in the body text matches the corresponding oscilloscope readout.
- Every figure has a number, a caption naming the probed pins, the measured quantity and the measured value, and is referenced from the body text.
- The final LCD photograph clearly shows “A” on the working module.
- All exact manual sections, datasheet tables and external sources are included in the References section.